# LAB TASK - 8

**1. Greedy Approach: Implementation of Huffman code**

**CODE:**

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

struct Node {
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f) {
        ch = c;
        freq = f;
        left = right = nullptr;
    }
};

struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

void printCodes(Node* root, string code) {
    if (root == nullptr)
        return;
```

```
    if (root->left == nullptr &&
        root->right == nullptr) {
        cout << root->ch << " : " << code << '\n';
        return;
    }

    printCodes(root->left, code + "0");
    printCodes(root->right, code + "1");
}

int main() {
    int n;
    cout << "Enter number of characters: ";
    cin >> n;

    if (n <= 0) {
        cout << "Invalid number of characters.\n";
        return 0;
    }

    priority_queue<Node*, vector<Node*>, Compare> pq;

    cout << "Enter character and frequency:\n";

    for (int i = 0; i < n; i++) {
        char ch;
        int freq;
        cin >> ch >> freq;

        if (freq <= 0) {
            cout << "Frequency must be positive.\n";
```

```
            return 0;
        }

        pq.push(new Node(ch, freq));
    }

    while (pq.size() > 1) {
        Node* left = pq.top();
        pq.pop();

        Node* right = pq.top();
        pq.pop();

        Node* parent = new Node(
            '\0', left->freq + right->freq
        );

        parent->left = left;
        parent->right = right;

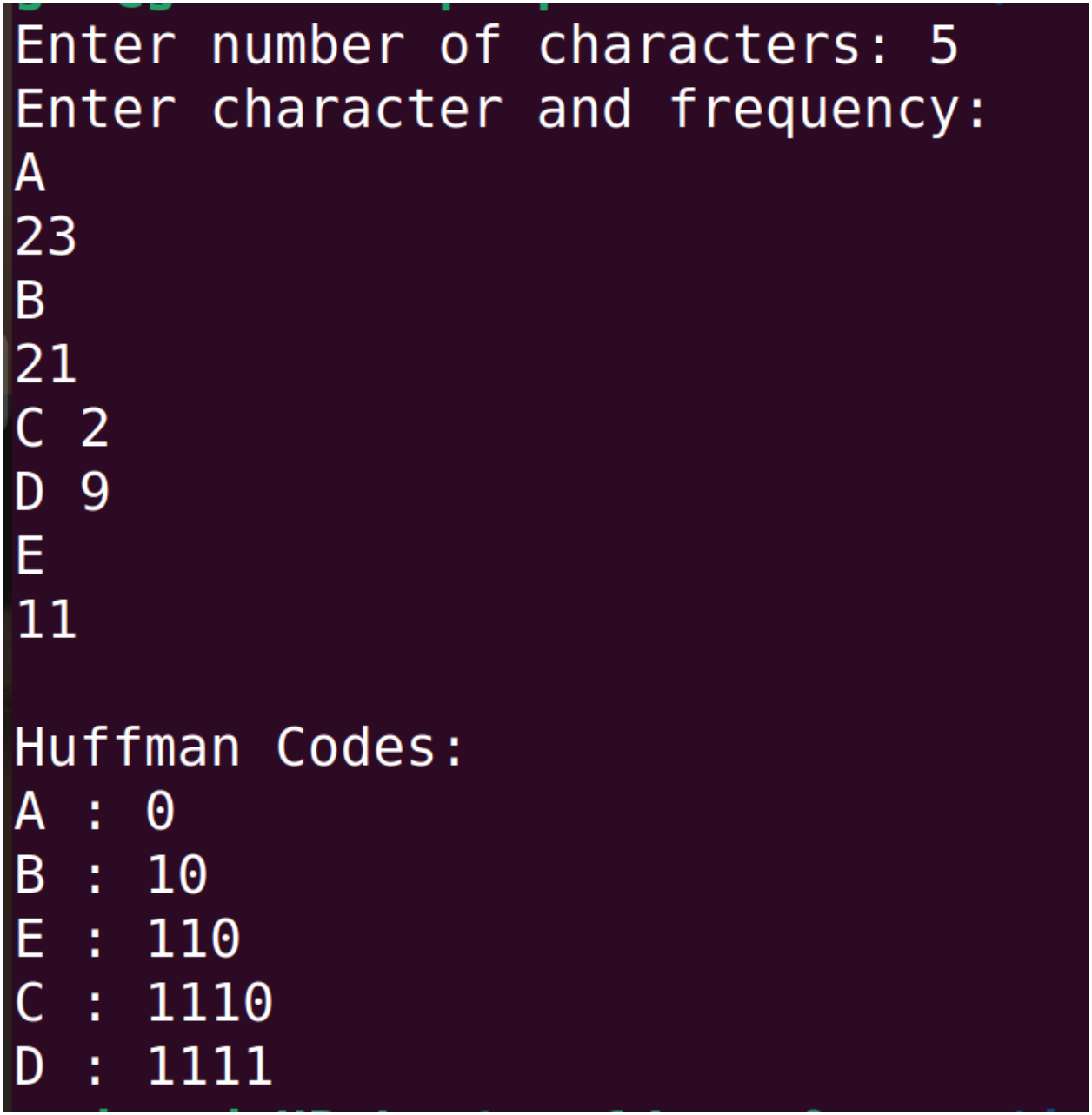
        pq.push(parent);
    }

    cout << "\nHuffman Codes:\n";

    if (n == 1)
        cout << pq.top()->ch << " : 0\n";
    else
        printCodes(pq.top(), "");

    return 0;
}
```

**OUTPUT:**


Enter number of characters: 5
Enter character and frequency:
A
23
B
21
C 2
D 9
E
11
Huffman Codes:
A : 0
B : 10
E : 110
C : 1110
D : 1111

**2. Assume that the numbers given below represent counts of letters in the hundreds from a file (similar to the CLRS example). For example, in the file there will be exactly 20 * 100 occurrences of the letter 'a' , 11*100 occurrence of the letter 'c', etc. a: 20, c:11, d:2, e: 10, o:15, m:8, s:10, t:22, u: 2**

**a) What is an optimal Huffman code based on the following set of frequencies?**

**1) Draw the tree. Show your work at every step.**

**2) Fill in the table on the right the Huffman encoding for each letter.**

**3) We encode the file using the Huffman codes produced above. How much memory will the file require with this encoding?**

**CODE:**

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;

struct Node {
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f) {
        ch = c;
        freq = f;
        left = right = nullptr;
    }
};

struct Compare {
```

```
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

// Display Huffman codes and calculate total bits
void printCodes(Node* root, string code, int& totalBits) {
    if (root == nullptr)
        return;

    // Leaf node represents a character
    if (root->left == nullptr && root->right == nullptr) {
        cout << root->ch << "\t"
             << root->freq << "\t"
             << code << "\t"
             << code.length() << endl;

        totalBits += root->freq * code.length();
        return;
    }

    printCodes(root->left, code + "0", totalBits);
    printCodes(root->right, code + "1", totalBits);
}

// Build Huffman tree
Node* buildHuffmanTree(char chars[], int freq[], int n) {
    priority_queue<Node*, vector<Node*>, Compare> pq;

    for (int i = 0; i < n; i++)
        pq.push(new Node(chars[i], freq[i]));
```

```
    while (pq.size() > 1) {
        Node* left = pq.top();
        pq.pop();

        Node* right = pq.top();
        pq.pop();

        Node* parent = new Node('\0',
                                left->freq + right->freq);

        parent->left = left;
        parent->right = right;

        pq.push(parent);
    }

    return pq.top();
}

int main() {
    char chars[] = {'a', 'c', 'd', 'e', 'o',
                    'm', 's', 't', 'u'};

    int freq[] = {20, 11, 2, 10, 15, 8, 10, 22, 2};

    int n = sizeof(chars) / sizeof(chars[0]);

    Node* root = buildHuffmanTree(chars, freq, n);

    int totalBits = 0;

    cout << "Character\tFrequency\tCode\tLength\n";
```

```
    cout << "----------------------------------------\n";

    printCodes(root, "", totalBits);

    cout << "\nTotal frequency units: 100\n";
    cout << "Total bits per 100 occurrences: "
         << totalBits << endl;

    cout << "Total bits for the file: "
         << totalBits * 100 << endl;

    cout << "Total bytes for the file: "
         << (totalBits * 100) / 8 << endl;

    return 0;
}
```

**OUTPUT:**

